



Date: 10 March 8, 2026

Software Requirement Specification (SRS)

PSB Hackathon 2026

**Software Requirement Specification
Version 1**

Project Name: Quantum-Proof Systems Scanner

Team Name: Neural Ninjas

Institute Name:
Indian Institute of Information Technology Kottayam



Revision History

Version No	Date	Prepared by/Modified by	Significant Changes
Draft V1.0			

Declaration

The purpose of this Software Requirements Specification (SRS) document is to identify and document the user requirements for the **Quantum-Proof Systems Scanner**. The end deliverable software that will be supplied by **Neural Ninjas** will comprise of all the requirements documented in the current document and will be operated in the manner specified in the document. The Source code will be developed subsequently based on these requirements and will formally go through code review during testing process.

Mentor Details (if any):

Indian Institute of Information Technology Kottayam

Signature:

Date:

Dr. Syamala S.

Assistant Professor

Email: syamala@iiitkottayam.ac.in

Team Member Details:

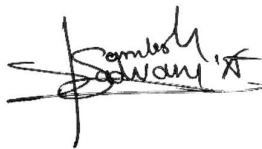
Member 1	Member 2	Member 3
IIIT Kottayam	IIIT Kottayam	IIIT Kottayam
		
Date: 10/03/2026	Date: 10/03/2026	Date: 10/03/2026
Kamlesh Malviya	Vijay Antony Rinish V	Prince Lavania
Team Lead	Developer	Tester



Table of Content

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Intendent Audience

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 Assumptions and Dependencies

3. Specific Requirements

- 3.1 Functional Requirements
- 3.2 External Interface Requirements
 - 3.2.1 User Interfaces
 - 3.2.2 Hardware Interfaces
 - 3.2.3 Software/ Communication Interfaces
- 3.3 System Features
- 3.4 Non-functional Requirements
 - 3.4.1 Performance Requirements
 - 3.4.2 Software Quality Attributes
 - 3.4.3 Other Non-functional Requirements

4. Technological Requirements

- 4.1 Technologies used in development of the web application
- 4.2 I.D.E. (Integrated Development Environment)
- 4.3 Database Management Software

5. Security Requirements



Software Requirement Specification (SRS)

❖ Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to identify and document the user requirement for **Quantum-Proof Systems Scanner**. This document is prepared with the following objectives:

- To provide behaviour of the system
- To provide Process Flow charts

1.2 Scope

- Discover cryptographic inventory (TLS certificates, VPN endpoints, APIs).
- Identify cryptographic controls (cipher suites, key exchange mechanisms, TLS versions).
- Validate whether deployed algorithms are quantum-safe.
- Generate actionable recommendations for non-PQC ready assets.
- Issue digital labels: *Quantum-Safe*, *PQC Ready*, or *Fully Quantum Safe*.
- Enterprise wide console for Central management: A GUI console to display status of scanned systems (public facing applications) covering details mentioned in **Appendix-A (Cert-In CBOM Elements)**. As per the variation of score (like High, Medium, Low rating etc) for any public applications, dashboard should display that change as well.

1.3 Intended Audience

The intended audience of this document is business and technical users from PNB.

❖ Overall Description

2.1 Product Perspective

The Quantum-Proof Systems Scanner will operate as an enterprise security assessment tool designed to analyze cryptographic implementations in public-facing banking applications.



The system will perform automated TLS handshake inspection and cryptographic analysis to determine whether systems are vulnerable to future quantum attacks.

The platform consists of the following modules:

- Asset Discovery Module
 - TLS Scanner Engine
 - Cryptographic Analysis Engine
 - Post-Quantum Validation Engine
 - Reporting and Dashboard System

2.2 Product Functions

- Scan public-facing domains and IP addresses.
- Extract TLS certificate metadata.
- Identify cipher suites and key exchange algorithms.
- Determine TLS protocol versions.
- Compare algorithms against NIST PQC standards.
- Generate CBOM inventory reports.
- Assign quantum readiness risk scores.
- Provide dashboard-based monitoring.

2.3 User Classes and Characteristics

Examples:

- **Primary Users:** Bank cybersecurity teams, IT administrators.
- **Secondary Users:** Compliance auditors, risk managers. Users are expected to have technical knowledge of cryptography and networking.

User at	User Type	Menus for User
PNB / IIT Kanpur officials	Admin User	
PNB	Checker	

2.4 Operating Environment

The operating environment for the '<project name>' as listed below.

- Server system
- Operating system:
- Database:
- Platform:
- Technology:
- API:



2.5 Design and Implementation Constraints

1. Technical Constraints: - (For Deployment)

- **Network Configuration:** e.g. The application must support private (intranet) IP addressing. Appropriate firewalls and routing rules must be configured.
- **Hosting Environment:** e.g. Should deploy in intranet i.e. for intranet access.

2. Security Constraints

- **Access Control:** e.g. (RBAC)
- **Data Encryption:** e.g. all data transmitted over the internet must be used by HTTPS.

3. Performance Constraints

- **Failover Mechanisms:** e.g. Ensure redundancy and failover mechanisms are in place for both environments to maintain availability.

4. User Interface Constraints

- **User Experience Consistency:** e.g. Maintain consistent design and navigation elements across both environments to minimize confusion for users switching between them.

Examples:

- Must comply with NIST PQC standards.
- Must operate only on public-facing applications.
- Must not disrupt live banking services.
- Must generate reports in machine-readable formats (JSON, XML, CSV).

2.6 Assumptions and Dependencies

- **Assumptions:**

Standard Browser Support- e.g. It is assumed that end users will be accessing the application using HTML5-compliant browser such as Google Chrome.

- **Dependencies:**

Database System-e.g. The application is dependent on Oracle database for data storage. Any maintenance, downtime, or performance issues with the database will directly impact on the application's functionality.



Examples:

- Assumes TLS-based communication is used in all public-facing applications.
- Depends on NIST PQC algorithms being standardized and available.
- Assumes internet connectivity for scanning endpoints.

❖ Specific Requirements

3.1 Functional Requirements

FR1: System shall allow administrators to submit target domains or IP addresses.

FR2: System shall perform TLS handshake analysis.

FR3: System shall extract cryptographic parameters including cipher suites and key exchange algorithms.

FR4: System shall retrieve certificate metadata such as issuer, subject, and validity period.

FR5: System shall analyze cryptographic algorithms against PQC standards.

FR6: System shall generate CBOM inventory reports.

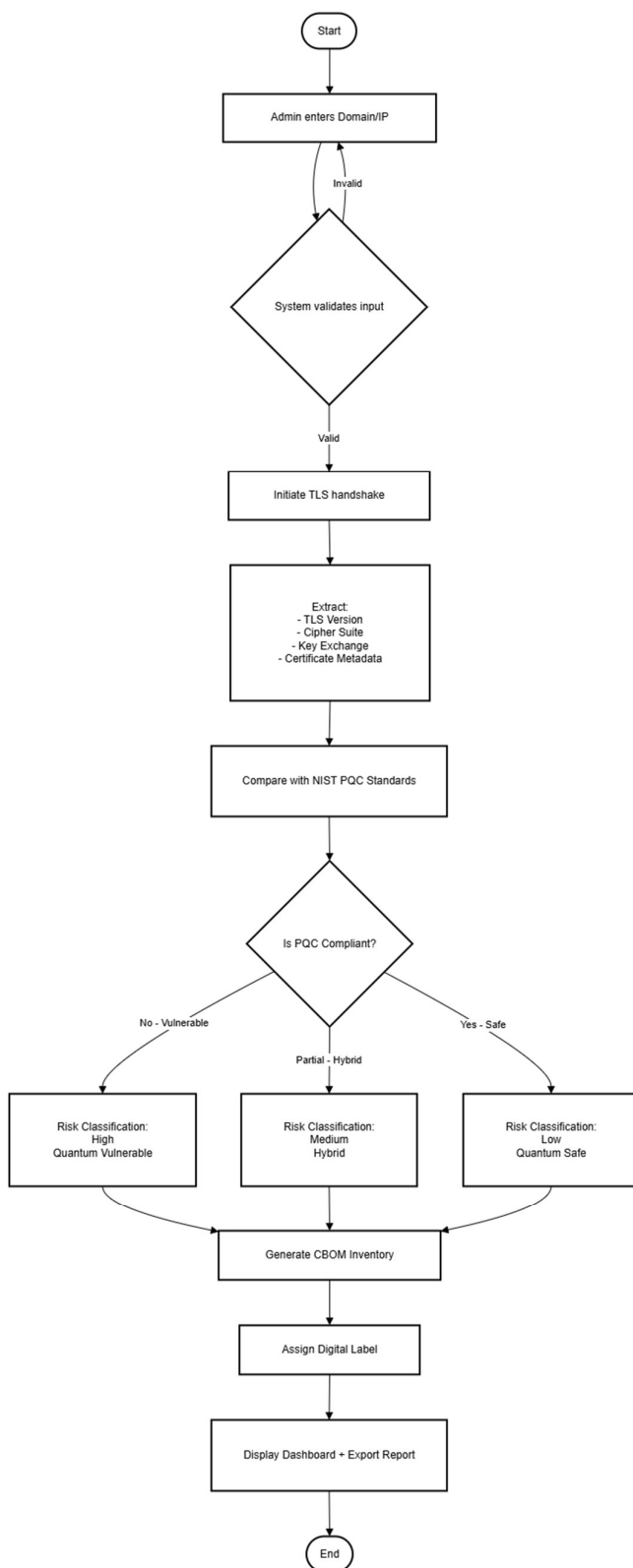
FR7: System shall classify assets into risk categories (High / Medium / Low).

FR8: System shall display results through a centralized dashboard.



WORKFLOW DAIGRAM (Process Flow)

Quantum Readiness Assessment Workflow



3.2 External Interface Requirements

3.2.1 User Interfaces

e.g. The application shall provide a web-based user interface accessible via a web browser - Google Chrome.

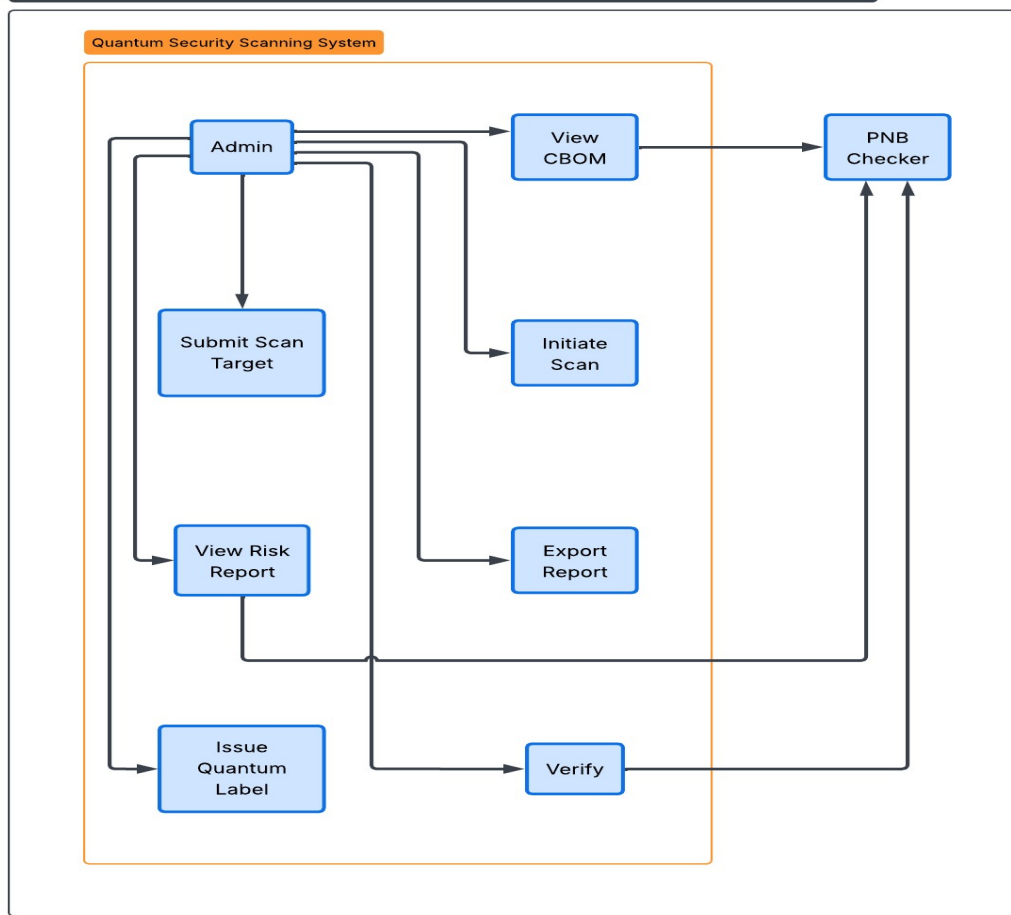
3.2.2 Hardware Interface

The system shall operate on enterprise servers capable of supporting network scanning operations.

3.2.3 Software Interface

- REST APIs
- TLS communication protocols
- Database connectivity

Quantum Security Scanning Use Case Diagram



3.3 System Features

- REST APIs
- TLS communication protocols
- Database connectivity

Level 0 DFD (Context Diagram)

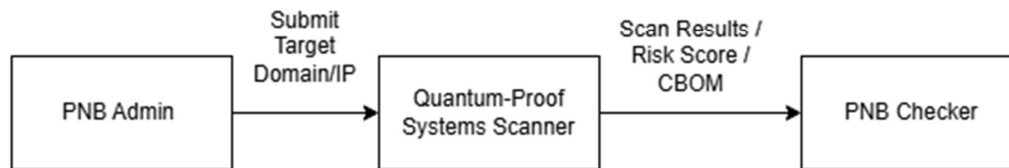


Figure 3: Data Flow Diagram Level 0

Level 1 DFD

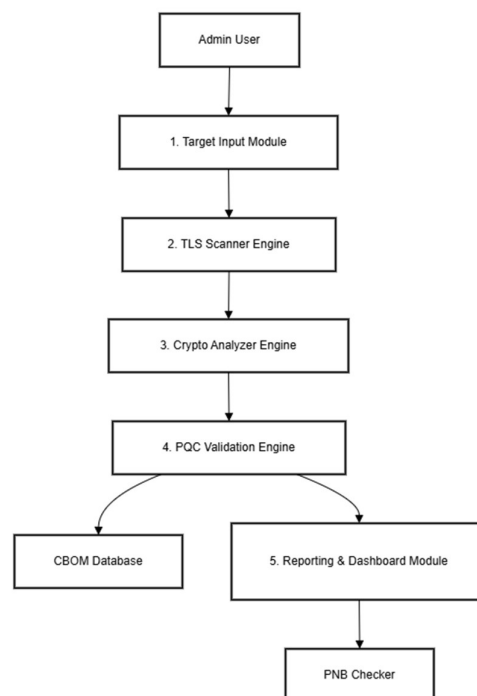


Figure 4: Data Flow Diagram Level 1

3.4 Non-functional Requirements

3.4.1 Performance Requirements

▪ Software Quality Attributes

3.4.3 Other Non-functional Requirements

Enterprise Architecture – Quantum-Proof Scanner

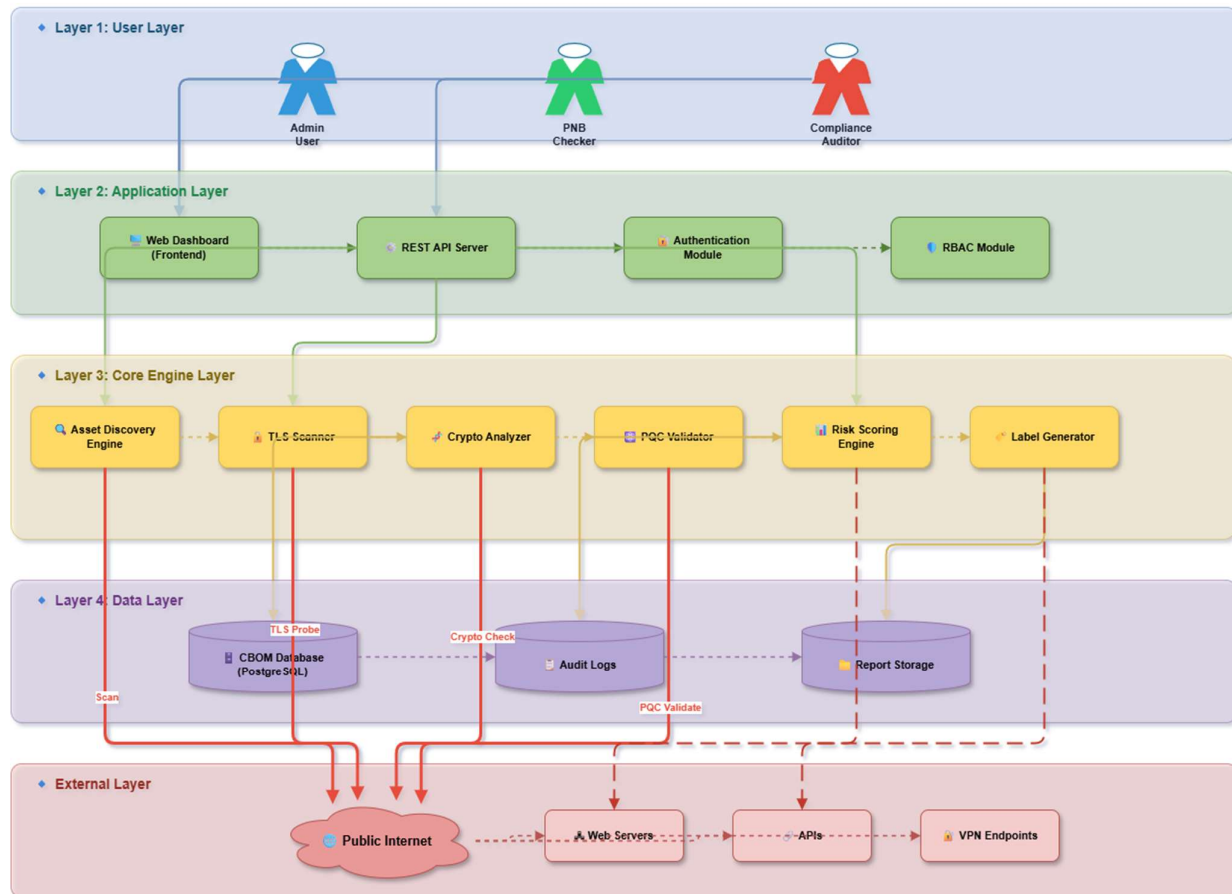


Figure 5 – Enterprise Architecture Diagram



4. Technological Requirements

4.1 Technologies used in development of the web application: e.g. Java, JSP, Servlet or other

Backend Framework: Python (Flask / Django)

- **Frontend Interface:** HTML5, CSS3, JavaScript
- **Cryptographic Analysis:** OpenSSL and Python SSL libraries
- **API Communication:** RESTful APIs
- **Containerization (Optional):** Docker for deployment

SS

4.2 I.D.E. (Integrated Development Environment): e.g. Eclipse or other.

- **Visual Studio Code (VS Code)** for backend and frontend development
- **PyCharm** for Python development and debugging
- **Draw.io / Lucidchart** for system architecture and UML diagram design

4.3 Database Management Software: e.g. Oracle SQL or other.

- **Database System:** PostgreSQL
- **Data Storage:** CBOM records, scan results, asset metadata
- **Data Formats Supported:** JSON, CSV, and structured relational tables

5. Security Requirements

The following points shall be considered at a minimum while preparing the security requirements for the system or system application:

- **Compatibility of the proposed system with current IT set up. Impact on existing systems should be estimated**

e.g. Existing system would not be affected.

- **Audit Trails for all important events capturing details like user ID, time and date, event etc.**

e.g. All the responses received from API are logged in DB

- **Control Access to Information and computing facilities based on principals like 'segregation of duty', 'need-to-know', etc**

e.g. Only Admin user will be able to schedule the application

- **Recoverability of Application in case of Failure**

e.g. Will be recovered from DR

- ❖ **Compliance with any legal, statutory and contractual obligations**

- ❖ **Security vulnerabilities involved when connecting with other systems and applications**

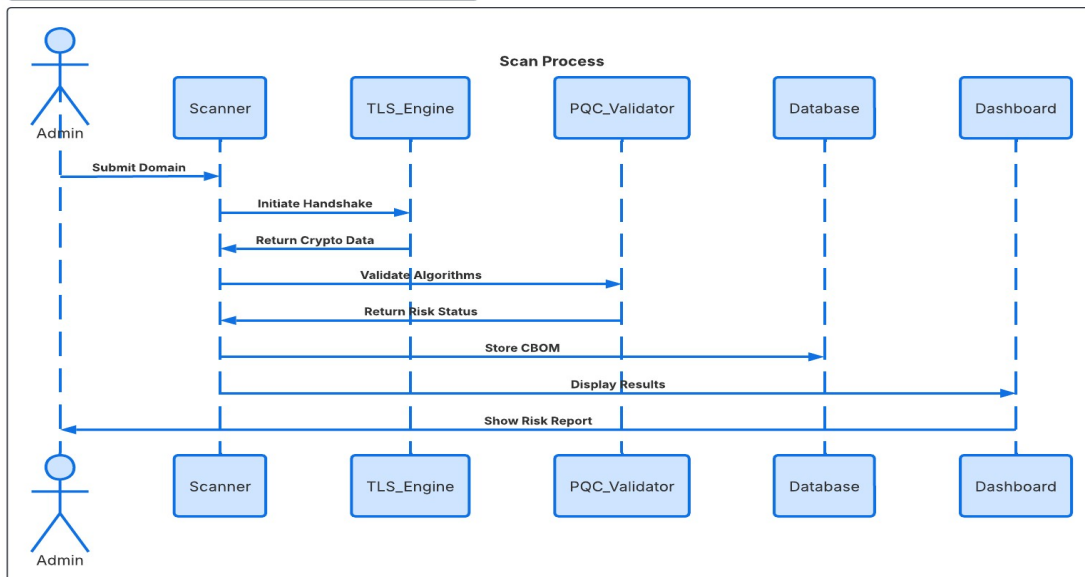
e.g. Will be found during Audit

- ❖ **Operating environment security**

e.g. TLS 1.2

- ❖ **Cost of providing security to the system over its life cycle (includes hardware, software, personnel and training).**

Scan Process Sequence Diagram





ਪੰਜਾਬ ਨੈਸ਼ਨਲ ਬੈਂਕ
...भरोसे का प्रतीक !



punjab national bank
...the name you can BANK upon !

Annexure-A (CERT-IN CBOM elements)



Table 9 : Minimum Elements pertaining to Cryptographic Asset

Cryptographic Asset Type	Element	Description
Algorithms	Name	The name of the cryptographic algorithm or asset. For example, "AES-128-GCM" refers to the AES algorithm with a 128-bit key in Galois/Counter Mode (GCM).
	Asset Type	Specifies the type of cryptographic asset. For algorithms, the asset type is "algorithm".
	Primitive	Describes the cryptographic primitive. For "SHA512withRSA", the primitive is "signature" as it's used for digital signing.
	Mode	The operational mode used by the algorithm. For example, "gcm" refers to the Galois/Counter Mode used with AES encryption.

Cryptographic Asset Type	Element	Description
	Crypto Functions	The cryptographic functions supported by the asset. For example, the functions in the case of "AES-128-GCM" are key generation, encryption, decryption, and authentication tag generation.
	Classical security level	The classical security level represents the strength of the cryptographic asset in terms of its resistance to attacks using classical (non-quantum) methods. For AES-128, it's 128 bits.
	OID	The Object Identifier (OID) is a globally unique identifier used to refer to the algorithm. It helps in distinguishing algorithms across different systems. For example, "2.16.840.1.101.3.4.1.6" for AES-128-GCM, "1.2.840.113549.1.1.13" for SHA512withRSA.
	List	Lists the cryptographic algorithms employed by the quantum device or system, allowing for an assessment of its security capabilities, especially in the context of post-quantum encryption standards.
Keys	Name	The name of the key, which is a unique identifier for the key used in cryptographic operations.
	Asset Type	Defines the type of cryptographic asset. For keys, the asset type is typically "key".
	id	A unique identifier for the key, such as a key ID or reference number.
	state	The state of the key, such as whether it is active, revoked, or expired.
	size	The size of the key, typically measured in bits. For example, a 128-bit key or a 2048-bit RSA key.
	Creation Date	The date when the key was created.
	Activation Date	The date when the key became operational or was first used.
Protocols	Name	The name of the cryptographic protocol, such as TLS, IPsec, or SSH



Cryptographic Asset Type	Element	Description
	Asset Type	Defines the type of cryptographic asset. In this case, it would be a "protocol"
	Version	The version of the protocol used, such as TLS 1.2 or TLS 1.3.
	Cipher Suites	The set of cryptographic algorithms and parameters supported by the protocol for tasks like encryption, key exchange, and integrity checking.
	OID	The Object Identifier (OID) associated with the protocol, identifying its unique specifications.
Certificates	Name	The name of the certificate, typically referring to its subject or the entity it represents (e.g., a website).
	Asset Type	Defines the type of cryptographic asset. For certificates, the asset type is "certificate".
	Subject Name	This refers to the Distinguished Name (DN) of the entity that the certificate represents. It typically contains information about the organization, domain name
	Issuer Name	The issuer is the Certificate Authority (CA) that issued and signed the certificate. This field contains the DN of the CA that verified and issued the certificate.
	Not Valid Before	This specifies the date and time from which the certificate is valid.
	Not Valid After	This specifies the expiration date and time of the certificate. The certificate becomes invalid after this timestamp.
	Signature Algorithm Reference	This refers to the cryptographic algorithm used to sign the certificate. It provides a reference to the algorithm and its OID (Object Identifier).
	Subject Public Key Reference	This points to the public key used by the subject (the entity being identified in the certificate). It provides a reference to the key's details, including the algorithm.
	Certificate Format	Specifies the format of the certificate. Common formats include X.509, which is the most widely used format for certificates.
Cryptographic Asset Type	Element	Description
	Certificate Extension	This refers to the file extension associated with the certificate. It is commonly .crt for certificates in the X.509 format.